



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
KATEDRA ZA AUTOMATIKU I UPRAVLJANJE SISTEMIMA

Simulacija upotrebom Julija paketa „Control Systems“

Modeliranje i simulacija sistema

Upravljanje, modelovanje i simulacija sistema

Julijin paket *ControlSystems*

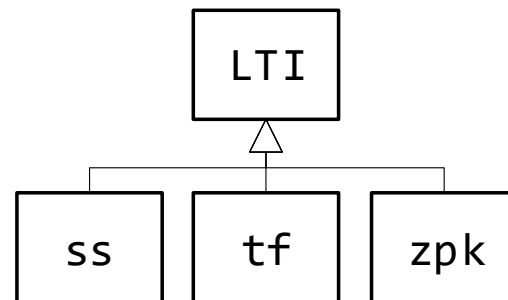
- Upotrebljava se za **modelovanje, analizu (simulacije)** i projektovanje sistema automatskog upravljanja.
- **Modeluje linearne vremenski nepromenljive modele**
- **Jedna promenljiva** se upotrebljava da opiše model sistema.
- Sadrži metode za analizu ponašanja modela - simulacije.
- Paket je napravljen po uzoru na Matlab biblioteku *Control System Toolbox*.

Ovaj tekst je napisan sa namerom da ilustruje samo najbitnije mogućnosti ovog paketa tako da neke od dodatnih funkcionalnosti nisu ni navedene.

Takođe, neke od funkcija koje se ovde spominju imaju dodatne oblike i/ili parametre, ali su njihovi opisi izostali da bi se fokus zadržao na bitnim mogućnostima.

Tipovi modela

- Podržani su linearnih vremenski nepromenljivi modeli opisani kao:
 - model u prostoru stanja - tipa `ss`
 - funkcija prenosa opisana količnikom polinoma - tipa `tf`
 - funkcija prenosa opisana preko nula/polova/pojačanja - tipa `zpk`
- Vremenski diskretni modeli imaju dodatni parametar - vreme odabiranja T_s .
- Model se opisuje jednim objektom (promenljivom)
- Svi navedeni tipovi objekata su izvedeni iz LTI (*Linear Time Invariant*) tipa objekta i sa njime dele neke zajedničke osobine.



Model u prostoru stanja - ss

Vremenski kontinualan

$$\dot{x}(t) = Ax(t) + Bu(t)$$

$$y(t) = Cx(t) + Du(t)$$

```
A = [0 1; 0 -12.5]
```

```
B = [0; 38.9]
```

```
C = [0 1]
```

```
D = 0
```

```
m = ss(A,B,C,D)
```

Vremenski diskretan

$$x(k+1) = Ex(k) + Fu(k)$$

$$y(k) = Cx(k) + Du(k)$$

```
# perioda odabiranja
```

```
Ts = 0.025
```

```
E = [1 0.02147  
      0 0.73160]
```

```
F = [0.01098  
      0.83520]
```

```
C = [0 1]
```

```
D = 0
```

```
md = ss(E,F,C,D,Ts)
```

Pristup parametrima modela:

```
julia> m.A           # ili m.B, m.C, m.D
2×2 Array{Float64,2}:
 0.0   1.0
 0.0 -12.5

julia> md.A
2×2 Array{Float64,2}:
 1.0  0.02147
 0.0  0.7316

julia> md.E
ERROR: type StateSpace has no field E
Stacktrace:
 [1] getproperty(::Any, ::Symbol) at .\Base.jl:20
 [2] top-level scope at REPL[20]:1

julia> m.Ts
0.0

julia> md.Ts
0.025
```

Funkcija prenosa (1)

Količnik polinoma

$$W(s) = \frac{p_n s^m + \dots + p_1 s + p_0}{s^n + \dots + q_1 s + q_0}$$

Ili

$$W(z) = \frac{p_n z^m + \dots + p_1 z + p_0}{z^n + \dots + q_1 z + q_0}$$

Upotreba Laplasove promenljive:

Uvodi se promenljiva – Laplasov operator “s”
(s = tf("s")) ili “z” (z = tf("z",Ts))

Nadalje se mogu upotrebljavati operatori:
+, -, *, /, ^

```
P = 38.9
Q = [1, 12.5, 0]
hm = tf(P, Q)

Ts = 0.01
P = [0.001866428867460801, 0.0017902675241066035]
Q = [1.0, -1.8824969025845955, 0.8824969025845955]
hmd = tf(P, Q, Ts)
```

```
s = tf("s")
hm = 38.9/(s^2 + 12.5*s)

z = tf("z",0.01)
H3 = (0.001866428867460801*z +
0.0017902675241066035) /
(1.0*z^2 - 1.8824969025845955*z +
0.8824969025845955)
```

Funkcija prenosa (2)

Nule polovi i pojačanje

$$W(s) = k \frac{(s - z_1)(s - z_2) \dots (s - z_m)}{(s - p_1)(s - p_2) \dots (s - p_n)}$$

Ili

$$W(z) = k \frac{(z - z_1)(z - z_2) \dots (z - z_m)}{(z - p_1)(z - p_2) \dots (z - p_n)}$$

```
nule = []          # nema nula
polovi = [0, -12.5] # dva pola
pojacanje = 38.9
hm = zpk(nule, polovi, pojacanje)

Ts = 0.01
nule = [-0.9591940819808409]
polovi = [1.0, 0.8824969025845955]
pojacanje = 0.001866428867460801
hm = zpk(nule, polovi, pojacanje, Ts)
```

Pristup parametrima modela:

```
julia> z, p, k = zpkdata(m);

julia> z
1×1 Array{Array{Complex{Float64},1},2}:
 []

julia> p
1×1 Array{Array{Complex{Float64},1},2}:
 [-12.5 + 0.0im]

julia> k
1×1 Array{Float64,2}:
 38.9
```

Matrica funkcija prenosa

$$Y(s) = G(s) \cdot U(s)$$

$$G(s) = \begin{bmatrix} G_{11}(s) & G_{12}(s) & \dots & G_{1m}(s) \\ G_{21}(s) & G_{22}(s) & \dots & G_{2m}(s) \\ \dots & \dots & \dots & \dots \\ G_{r1}(s) & G_{r2}(s) & \dots & G_{rm}(s) \end{bmatrix}$$

- Uvodi se kao matrica objekata

```
julia> W = [tf(1,[2,3])      zpk([-1.0],[-2.0,-3.0],1.0)
            tf(12,[4,1,2])  tf(1,[1,0])]
TransferFunction{ControlSystems.SisoZpk{Float64,Complex{Float64}}}}
Input 1 to output 1
      1.0
0.5-----
      1.0*s + 1.5

Input 1 to output 2
      1.0
3.0-----
      1.0*s^2 + 0.25*s + 0.5

Input 2 to output 1
      1.0*s + 1.0
1.0-----
      (1.0*s + 2.0)(1.0*s + 3.0)

Input 2 to output 2
      1.0
1.0-----
      1.0*s

Continuous-time transfer function model

julia> size(W)          # dimenzija modela
(2, 2)

julia> W[1,2]          # pristup pojedinačnoj FP
TransferFunction{ControlSystems.SisoZpk{Float64,Complex{Float64}}}}
      1.0*s + 1.0
1.0-----
      (1.0*s + 2.0)(1.0*s + 3.0)

Continuous-time transfer function model
```

Konverzije modela

Funkcije (konstruktori modela) `tf`, `zpk` i `ss` se mogu upotrebiti da konvertuju ranije unet model u željeni tip.

```
julia> h = tf(1, [1, 2, 1])
TransferFunction{ControlSystems.SisoRational{Int64}}
      1
-----
s^2 + 2*s + 1
Continuous-time transfer function model

julia> h2 = zpk(h)
TransferFunction{ControlSystems.SisoZpk{Float64,Complex{Float64}}}
      1.0
-----
1.0-----
      (1.0*s + 1.0)(1.0*s + 1.0)
Continuous-time transfer function model

julia> w = tf(m)
TransferFunction{ControlSystems.SisoRational{Float64}}
      38.9*s
-----
1.0*s^2 + 12.5*s
Continuous-time transfer function model

julia> m2 = ss(w)
StateSpace{Float64,Array{Float64,2}}
A =
  0.0    1.0
 -0.0  -12.5
B =
  0.0
  1.0
C =
  0.0  38.9
D =
  0.0
Continuous-time state-space model
```


Vremenska diskretizacija

Konvertovanje vremenski kontinualnog u vremenski diskretan model vrši se pozivanjem funkcije `c2d` gde se zadaje vreme odabiranja T_s .

- Podrazumeva se da se na izlazu iz odabirača koristi kolo zadržke signala nultog reda (*zero-order hold*).

```
A = [0 1; 0 -12.5];
B = [0; 38.9];
C = [1 0; 0 1];      # model sa dva izlaza
D = 0;
m = ss(A,B,C,D);

md1 = c2d(m, 0.05)
md2 = c2d(m, 0.025)
```

```
julia> md1
(StateSpace{Float64,Array{Float64,2}}
A =
 1.0  0.03717908571848078
 0.0  0.5352614285189902
B =
 0.039898685244087816
 1.4462664344489022
C =
 1.0  0.0
 0.0  1.0
D =
 0.0
 0.0
Sample Time: 0.05 (seconds)
Discrete-time state-space model,
[1.0 0.0 0.0; 0.0 1.0 0.0])
```

```
julia> md2
(StateSpace{Float64,Array{Float64,2}}
A =
 1.0  0.02147074968426866
 0.0  0.7316156289466417
B =
 0.010983026982555943
 0.8352121627180507
C =
 1.0  0.0
 0.0  1.0
D =
 0.0
 0.0
Sample Time: 0.025 (seconds)
Discrete-time state-space model,
[1.0 0.0 0.0; 0.0 1.0 0.0])
```

```
julia> W = tf(38.9, [1, 12.5])
TransferFunction{ControlSystems.SisoR
ational{Float64}}
      38.9
-----
1.0*s + 12.5
```

Continuous-time transfer function model

```
julia> Wd1 = c2d(W, 0.05)
TransferFunction{ControlSystems...}
      1.4462664344489025
-----
1.0*z - 0.5352614285189903
```

Sample Time: 0.05 (seconds)
Discrete-time transfer function model

```
julia> Wd2 = c2d(W, 0.025)
TransferFunction{ControlSystems...}
      0.8352121627180507
-----
1.0*z - 0.7316156289466419
```

Sample Time: 0.025 (seconds)
Discrete-time transfer function model

Analiza ponašanja modela

- Analiza ponašanja modela se može sprovesti u vremenskom i u kompleksnom domenu.
- Analiza u vremenskom domenu se odnosi na simulacije, tj. na izračunavanje izlaza iz modela kada se pobudi željenim ulazom.
- Funkcije za izračunavanje izlaza su:
 - `step` - daje jedinični odziv (ulaz je Hevisajdova funkcija $h(t)$)
 - `impulse` - daje impulsni odziv (ulaz je Dirakov impuls $\delta(t)$)
 - `lsim` - daje odziv na ulaz zadat u vidu niza vrednosti.
Takođe omogućava da se definiše i početna vrednost promenljivih stanja x_0 .

step, impulse

- Odziv modela na jediničnu pobudu
`y,t,x = step(model, tu)`
- Odziv modela na impulsnu pobudu
`y,t,x = impulse(model, tu)`

Gde su:

- `tu` – opseg vremena ili krajnje vreme
- `t` – opseg vremena
- `y` – matrica vrednosti izlaza
vrsta je određena trenutkom iz `t`
kolona određuje redni broj izlaza
- `x` – matrica vrednosti prom. stanja
vrsta je određena trenutkom iz `t`
kolona određuje redni broj pr. stanja

```
W = tf(38.9, [1, 12.5]);  
md1 = c2d(W, 0.05);  
md2 = c2d(W, 0.025);
```

```
y,t,x = step(W, 5);  
y2,t2,_ = step(Wd1, 0.2);  
y3,t3,_ = step(Wd2, 0.2);
```

```
julia> t  
0.0:0.07:4.97
```

```
julia> y  
72×1 LinearAlgebra.Transpose{...}:  
 0.0  
 1.8147273301519837  
 2.57122104057078  
 2.8865731773531578  
 3.0180198979634323  
 ⋮  
 3.1114866868906805  
 3.1117835612260674
```

```
julia> t2  
0.0:0.05:0.2
```

```
julia> y2  
5×1 LinearAlgebra.Transpose{...}:  
 0.0  
 1.4462664344489025  
 2.2203970721710884  
 2.6347593431785827  
 2.856551484282427
```

```
julia> t3  
0.0:0.025:0.2
```

```
julia> y3  
9×1 LinearAlgebra.Transpose{...}:  
 0.0  
 0.8352121627180507  
 1.4462664344489025  
 1.8933232897818015  
 2.2203970721710884  
 2.459689363185784  
 2.634759343178583  
 2.762843276700691  
 2.8565514842824276
```

```
julia>
```

lsim

Odziv na prizvoljnu pobudu i početno stanje

`y, t, x = lsim(model, u, t; x0)`

Gde su:

- `t` – opseg vremena
- `u` – promene ulaza za trenutke `u` `t`
- `x0` – početne vrednosti promenljivih stanja
- `y` – matrica vrednosti izlaza
vrsta je određena trenutkom iz `t`
kolona određuje redni broj izlaza
- `x` – matrica vrednosti prom. stanja
vrsta je određena trenutkom iz `t`
kolona određuje redni broj pr. stanja

```
t = 0:0.5:2;  
u = [1.0, 2.0, 2.0, 1.5, 1.0];  
x0 = [1.0, 0.0];  
y, t, x = lsim(m, u, t, x0)
```

```
julia> W  
TransferFunction{ControlSystems.SisoRational{Float64}}  
      38.9  
-----  
1.0*s + 12.5  
  
Continuous-time transfer function model  
  
julia> t = 0:0.5:2;  
  
julia> u = [1.0, 2.0, 2.0, 1.5, 1.0];  
  
julia> y,t,x = lsim(W, u, t);  
  
julia> y  
5×1 LinearAlgebra.Transpose{Float64,Array{Float64,2}}:  
 36.28495878827649  
  0.5660458809038063  
  0.9930915898683375  
  0.9939159820235663  
  0.7459178574107492  
  
julia> t  
0.0:0.5:2.0
```

Složen model u simulacionom softveru

- Delovi modela su opisani promenljivima (objektima)
 - ss, zpk, tf, ...
- Na delove modela (promenljive) se mogu primeniti transformacije modela
 - algebra blokova funkcija prenosa je primenljiva na sve LTI tipove objekata (promenljivih)
- Postupno povezivanje delova modela:
 - redna veza: funkcija series ili operator množenja "*"
 - paralelna veza: funkcija parallel ili operator sabiranja "+" ili oduzimanja "-"
 - povratna sprega: funkcija feedback.

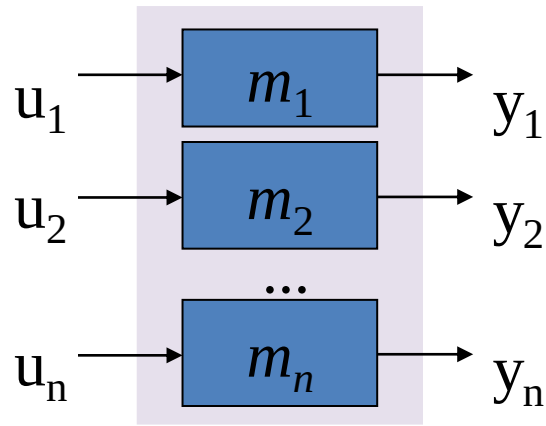
Pored navedenih, mogu se koristiti i operatori stepenovanja "^" i deljenja "/".
- Objedinjavanja delova u nepovezan model – append
 - Dobija se blok dijagonalna forma rezultujućeg modela

Postupno povezivanje delova modela

- **Paralelna veza:** povezuje dva modela u paralelnoj vezi. Modeli imaju zajedničke ulaze, a izlazi im se sabiraju/oduzimaju.
 - $m = \text{parallel}(m1, m2)$
 - $m = m1 + m2$ (ili operator „-“)
- **Redna veza:** povezuje dva modela u serijskoj vezi. Izlazi prvog modela se povezuju na ulaze drugog modela.
 - $m = \text{series}(m1, m2)$
 - $m = m1 * m2$
- **Povratna sprega:** povezuje dva modela u zatvorenu negativnu povratnu spregu (m1 je u direktnoj, a m2 u povratnoj grani)
 - $m = \text{feedback}(m1, m2)$
 - Alternativa: uneti izraz sa operatorima (uključujući i „/“)

append

- objedinjuje opis više modela u jedan model
 $m = \text{append}(m_1, m_2, \dots, m_n)$



Napomena:

U opštem slučaju, na slici i u formuli su ulazi $(x_1, x_2, \dots)$, izlazi $(y_1, y_2, \dots)$ i promenljive stanja $(x_1, x_2, \dots)$ vektori.

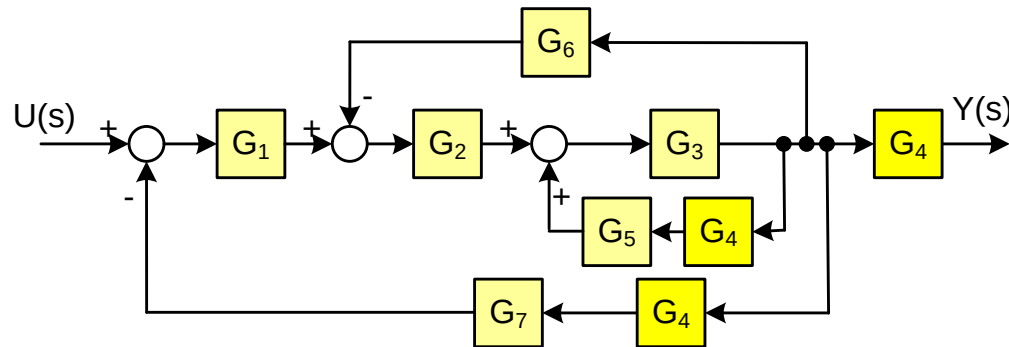
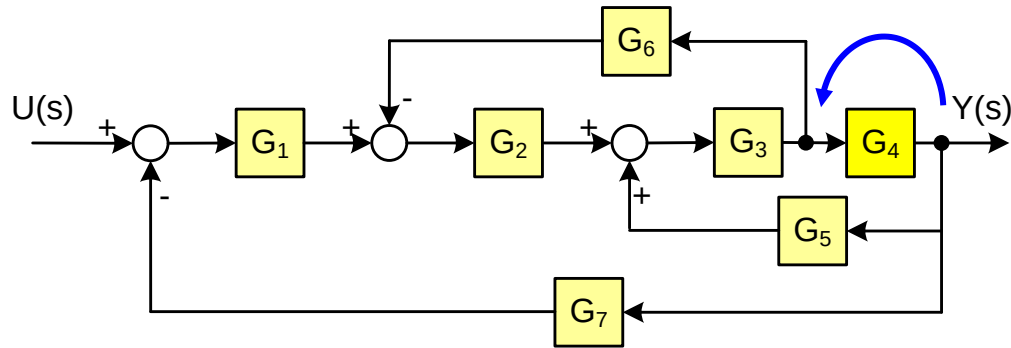
U vremenskom domenu (blok-dijagonalna forma):

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dots \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} A_1 & 0 & \dots & 0 \\ 0 & A_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & A_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} + \begin{bmatrix} B_1 & 0 & \dots & 0 \\ 0 & B_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & B_n \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \dots \\ u_n \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix} = \begin{bmatrix} C_1 & 0 & \dots & 0 \\ 0 & C_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & C_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} + \begin{bmatrix} D_1 & 0 & \dots & 0 \\ 0 & D_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & D_n \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \dots \\ u_n \end{bmatrix}$$

U kompleksnom domenu:

$$W(s) = \begin{bmatrix} H_1(s) & 0 & \dots & 0 \\ 0 & H_2(s) & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & H_n(s) \end{bmatrix}$$

Primer 1



```
g1 = tf(0.5);
g2 = tf(10,[1,2]);
g3 = tf(3.4/2);
g4 = tf(2,[1, 1, 2]);
g5 = tf([1, 1],[1, 2, 7]);
g6 = tf(1,[1, 2.5]);
g7 = tf(1/12);

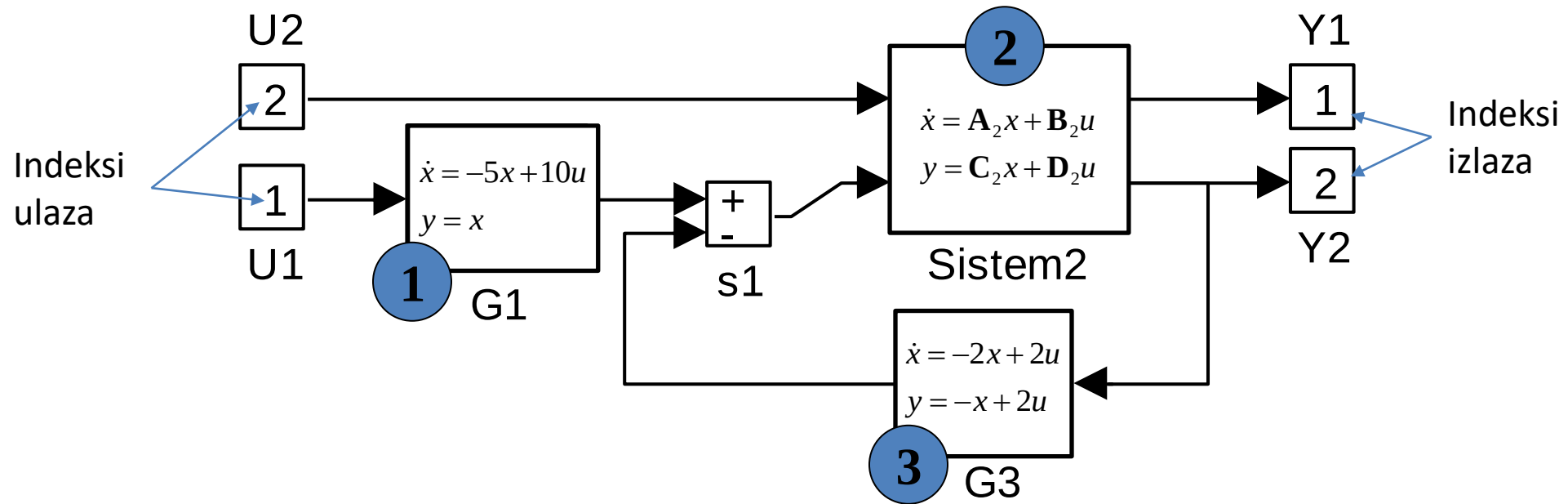
g = feedback(g3, -g4*g5);
g = feedback(g2*g, g6);
g = g4*feedback(g1*g, g7*g4);
g = minreal(g, 0.001)
```

Transfer function:

$$17 s^3 + 76.5 s^2 + 204 s + 297.5$$

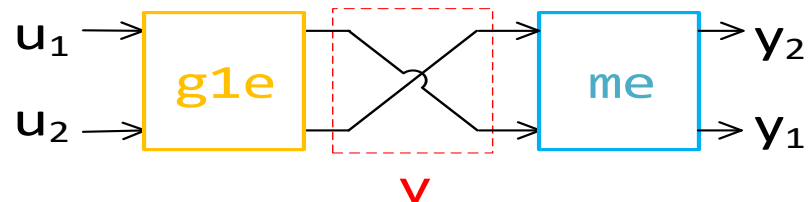
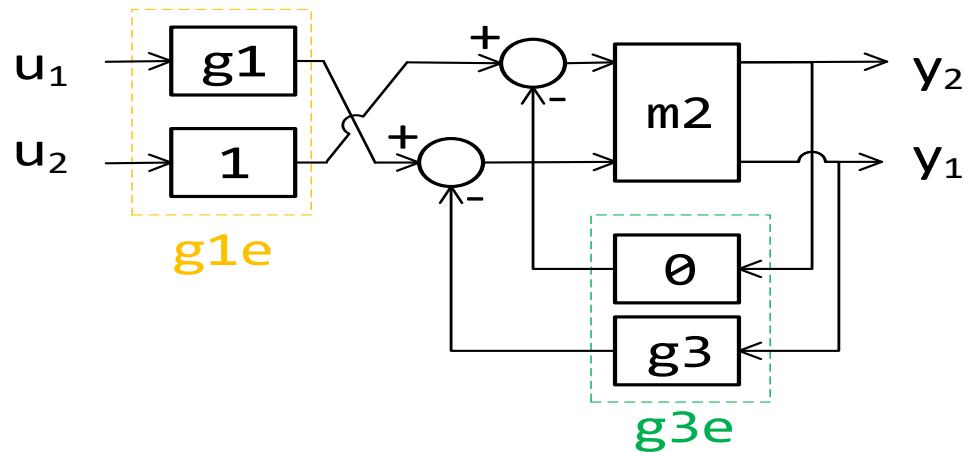
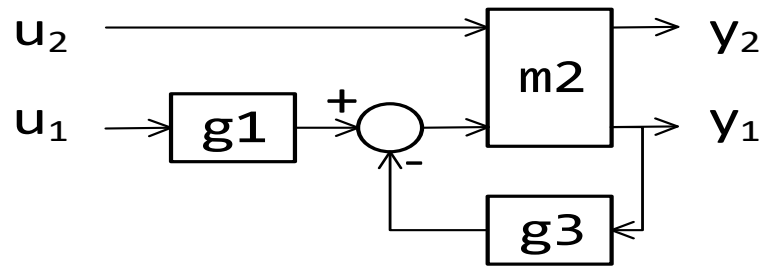
$$s^6 + 7.5 s^5 + 46.5 s^4 + 124.5 s^3 + 293.2 s^2 + 289.7 s + 315.8$$

Primer 2 - postavka



```
g1 = ss(-5,10,1,0);      # blok 1
a2 = [-9.02 17.78;  -1.69  3.21];
b2 = [-0.51  0.54;  -0.00 -1.84];
c2 = [-3.29  2.45; -13.50 18.07];
d2 = [-0.55 -0.14; -0.64  0.29];
m2 = ss(a2,b2,c2,d2);    # blok 2
g3 = ss(-2,2,-1,2);      # blok 3, matrica D <> 0!
...
```

Primer 2 - rešenje



```
# g1 = ...
```

```
# g3 se proširuje dodatnim ulazom i izlazom
g3e = append(ss(0), g3);
# zatvara se povratna sprega oko m2 preko g3e
me = myfeedback(m2, g3e);
```

```
# blok 1 se proširuje dodatnim ulazom i izlazom kao y2=u2
g1e = append(g1, ss(1));
```

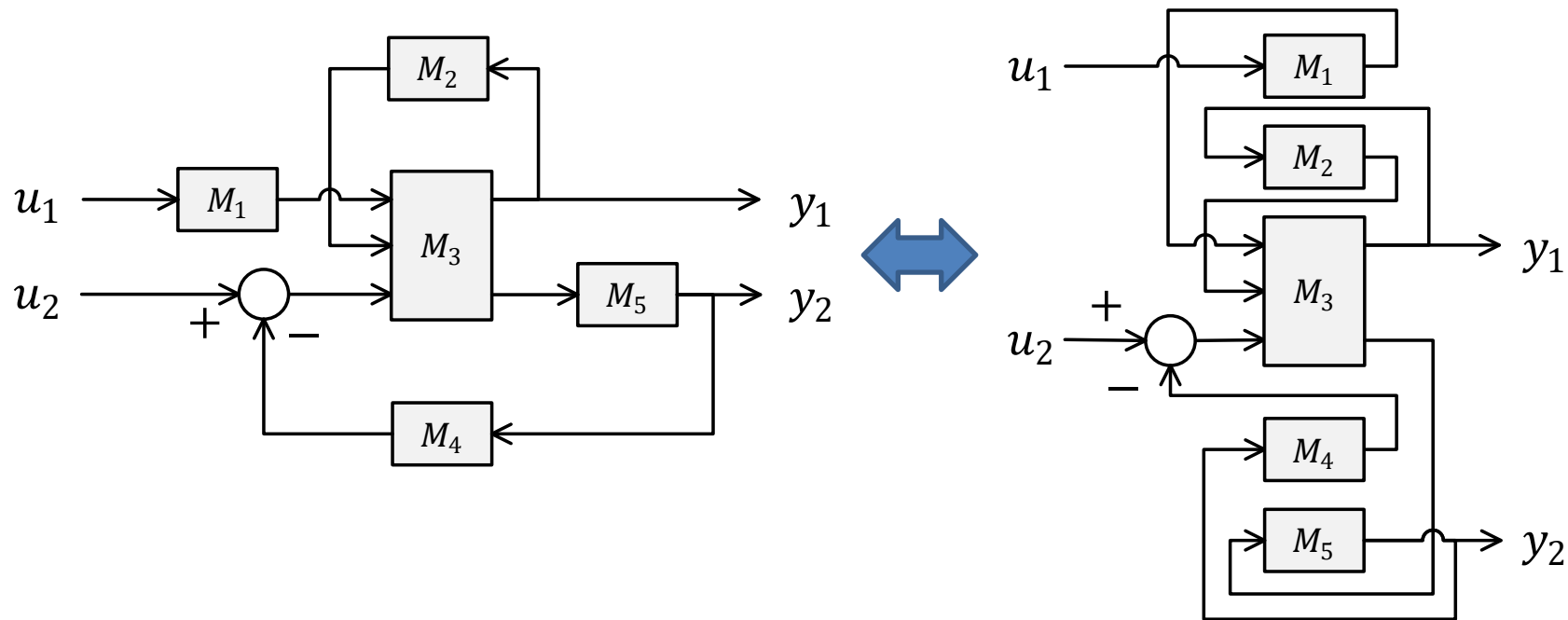
```
# model bloka sa "ukrštenim vezama": y1=u2, y2=u1
# model ima samo matricu pojačanja
v = ss([0 1; 1 0])
```

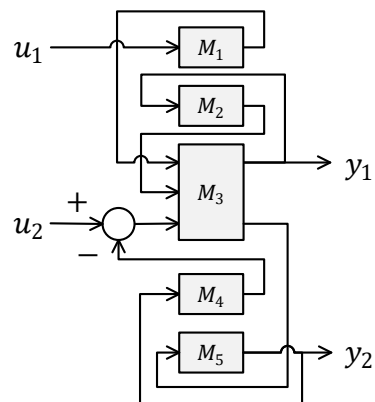
```
# serijske veze (prema donjoj slici)
mm = series(g1e, v);
m = series(mm, me)
```

Primer: povezivanje delova modela preko opisa veza

Princip:

- svi delovi modela se objedine u jedan nepovezan model M_d - *append*
 - veze delova modela se predstavljaju kao jedinične povratne sprege izlaza M_d na njegove ulaze – *connect*
- Napomena: *connect* funkcija nije deo paketa ControlSystems.

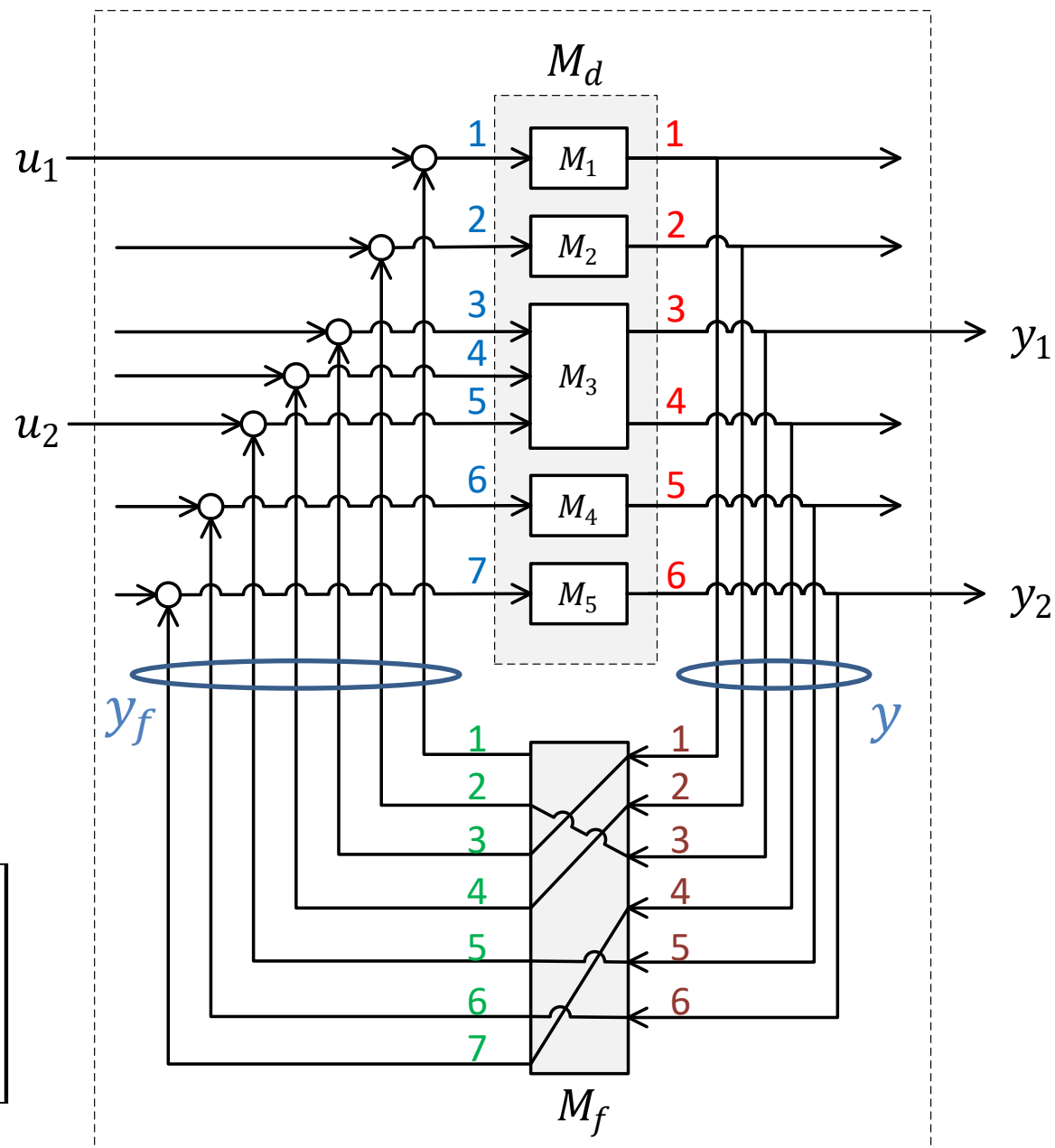




Opis M_f :

$$y_f = D_f \cdot y$$

$$\begin{bmatrix} y_{f1} \\ y_{f2} \\ y_{f3} \\ y_{f4} \\ y_{f5} \\ y_{f6} \\ y_{f7} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{bmatrix}$$



Transformacija *connect*-a

Opis modela:

- direktna grana:

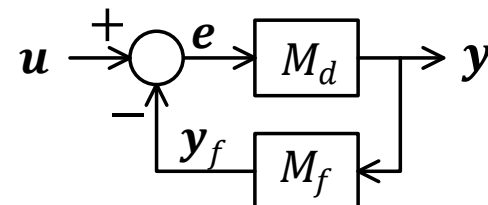
$$\dot{x}(t) = A_d x(t) + B_d e(t)$$

$$y(t) = C_d x(t) + D_d e(t)$$

- povratna grana:

$$y_f(t) = D_f y(t)$$

Kako napraviti D_f ? => Upotrebom matrice veza!



Ekvivalentan model:

$$\dot{x}(t) = Ax(t) + Bu(t)$$

$$y(t) = Cx(t) + Du(t)$$

gde su:

$$Q = (I + D_f D_d)^{-1}$$

$$B = B_d Q$$

$$D = D_d Q$$

$$A = A_d - B D_f C_d$$

$$C = C_d - D D_f C_d$$

dokaz



Izvođenje transformacije *connect-a*

$$e(t) = u(t) - y_f(t) = u(t) - D_f y(t) = u(t) - D_f (C_d x(t) + D_d e(t))$$

$$e(t) + D_f D_d e(t) = u(t) - D_f C_d x(t)$$

$$(I + D_f D_d) e(t) = u(t) - D_f C_d x(t)$$

$$e(t) = (I + D_f D_d)^{-1} u(t) - (I + D_f D_d)^{-1} D_f C_d x(t)$$

$$\dot{x}(t) = A_d x(t) + B_d \left((I + D_f D_d)^{-1} u(t) - (I + D_f D_d)^{-1} D_f C_d x(t) \right)$$

$$\dot{x}(t) = \left(A_d - B_d (I + D_f D_d)^{-1} D_f C_d \right) x(t) + B_d (I + D_f D_d)^{-1} u(t)$$

$$\dot{x}(t) = A x(t) + B u(t)$$

$$A = A_d - B_d (I + D_f D_d)^{-1} D_f C_d, \quad B = B_d (I + D_f D_d)^{-1}$$

$$y(t) = C_d x(t) + D_d \left((I + D_f D_d)^{-1} u(t) - (I + D_f D_d)^{-1} D_f C_d x(t) \right)$$

$$y(t) = \left(C_d - D_d (I + D_f D_d)^{-1} D_f C_d \right) x(t) + D_d (I + D_f D_d)^{-1} u(t)$$

$$y(t) = C x(t) + D u(t)$$

$$C = C_d - D_d (I + D_f D_d)^{-1} D_f C_d, \quad D = D_d (I + D_f D_d)^{-1}$$

Upotreba *connect-a*

- Formira model složenog sistema na osnovu poznatih blokova modela
 $m = \text{connect}(mo, veze, indul, indiz)$
- Koraci:
 - svi blokovi u blok dijagramu se označe rednim brojevima 1, 2 ...
 - prema uvedenom redosledu blokova napravi se objedinjeni model *mo* pozivom *append*
 - numerišu se monotonim nizom (1, 2, ...) svi ulazi, počevši od bloka sa indeksom 1 i nastavljajući rast indeksa na narednim blokovima
 - numerišu se izlazi svih blokova (1, 2, ...)
 - odrede se veze među blokovima – matrica veza (vidi naredni slajd)
 - odrede se indeksi ulaza *indul* i izlaza *indiz* rezultujućeg modela
 - pozove se funkcija *connect*

Veze u *connect*-u

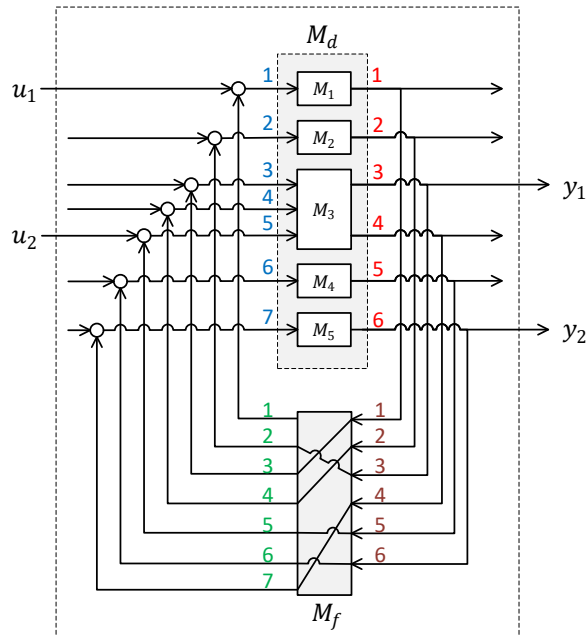
- Problem:
Kako jednostavno opisati veze između delova modela?
Tj. Kako jednostavno opisati mapiranje ulaza na izlaze u M_f ?
- Jedno rešenje:
Treba opisati svaku vezu preko indeksa ulaza $indu_k, k \in \{1, \dots, m\}$ i indeksa izlaza $indy_i, i \in \{1, \dots, r\}$, koji se povezuju na posmatrani ulaz:

$$veza = [indu_k \quad indy_{k,1} \quad indy_{k,2} \quad \dots]$$

- Definisanje svih veza formira matricu
(0 “dopunjava” vrstu gde ima manji broj izlaza)

$$veze = \begin{bmatrix} indu_1 & indy_a & indy_b & \dots \\ indu_2 & indy_c & 0 & \dots \\ \dots & \dots & \dots & \dots \\ indu_m & indy_d & indy_e & \dots \end{bmatrix}$$

Primer: veze... u *connect*-u



- Veze:

$$veze = \begin{bmatrix} 2 & 3 \\ 3 & 1 \\ 4 & 2 \\ 5 & 5 \\ 6 & 6 \\ 7 & 4 \end{bmatrix} \rightarrow D_f = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

- Indeksi uzaza i uzlaza rezultujućeg modela:
 - od 7 ulaza treba zadržati 1. i 5.
 - od 6 izlaza treba zadržati 3. i 6.

$$indul = [1 \quad 5]$$

$$indiz = [3 \quad 6]$$

Implementacija *connect*

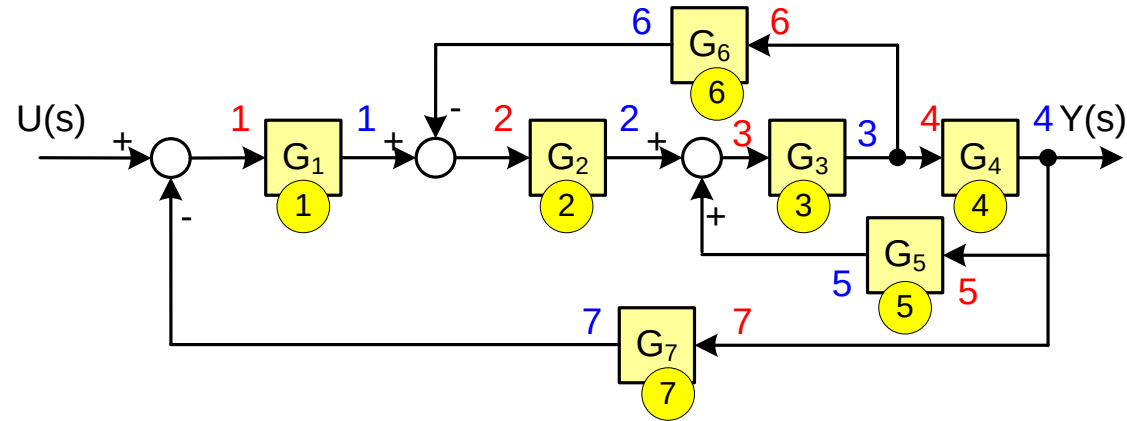
```
function connect(m, veze, indul, indiz)
    Md = ss(m)
    Df = zeros(Md.ny, Md.nu);           # formiranje matrice Df
    n,m = size(veze)
    for i in 1:n, j = 2:m
        k = veze[i,j]
        if k != 0
            Df[veze[i,1], abs(k)] = -sign(k)
        end
    end
    end

    # naredni kod radi u osnovi: Mf = ss(Df); M = feedback(Md,Mf)
    Ad, Bd, Cd, Dd = Md.A, Md.B, Md.C, Md.D

    Qi = Df * Dd + I
    B = Bd / Qi
    D = Dd / Qi
    A = Ad - B * Df * Cd
    C = Cd - D * Df * Cd

    C = C[indiz, :];    D = D[indiz, :]    # brisanje nepotrebnih izlaza
    B = B[:, indul];    D = D[:, indul]    # brisanje nepotrebnih ulaza
    return ss(A, B, C, D)
end
```

Primer 1 – *connect*

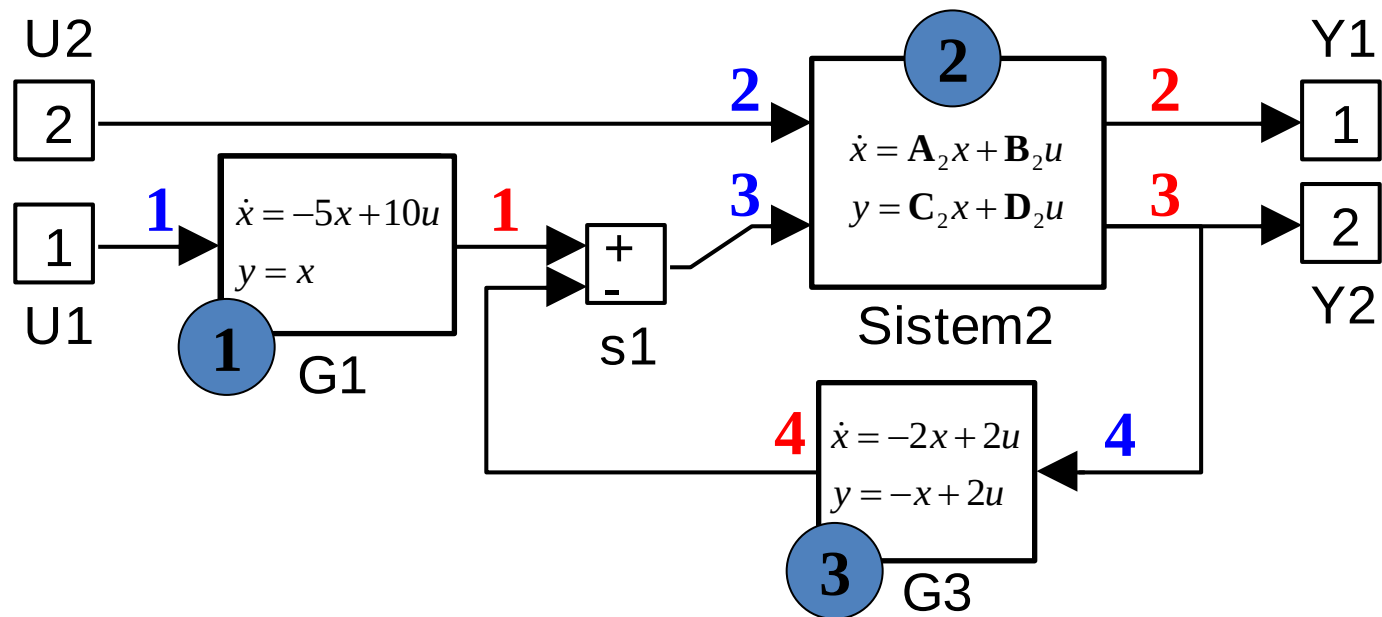


```
# g1=...
W = append(g1, g2, g3, g4, g5, g6, g7);
veze = [ 1 -7  0
         2  1 -6
         3  2  5
         4  3  0
         5  4  0
         6  3  0
         7  4  0];
ulazi = [1]; izlazi = [4];
m = connect(W, veze, ulazi, izlazi)
```

Transfer function:

$$\frac{17 s^3 + 76.5 s^2 + 204 s + 297.5}{s^6 + 7.5 s^5 + 46.5 s^4 + 124.5 s^3 + 293.2 s^2 + 289.7 s + 315.8}$$

Primer 2 - connect



```

g1 = ...
m2 = ...
g3 = ...

m = append(g1,m2,g3);
veze = [3 1 -4; 4 3 0]; # veze na osnovu uvedenih brojeva
ulazi = [1, 2];         # redni brojevi ulaza
izlazi = [2, 3];        # redni brojevi izlaza
q = connect(m,veze,ulazi,izlazi)
    
```